CSE411 Assignment #3: LL(1) Top-Down Parser

Sungho Kim (20241065)

October 12, 2026

1 The Parsing Table for *valid-grammar.json*

Nonterminal	INT	+	*	(	)	$
E	$E \to T\ E'$			$E \to T\ E'$		
E'		$E' \to +\ T\ E'$			$E' \to \varepsilon$	$E' \to \varepsilon$
T	$T \to F\ T'$			$T \to F\ T'$		
T'		$T' \to \varepsilon$	$T' \to *\ F\ T'$		$T' \to \varepsilon$	$T' \to \varepsilon$
F	$F \to$ INT			$F \to (\ E\ )$		

2 The Parsing Table for *invalid-grammar.json*

Nonterminal	INT	+	*	(	)	$
E	$E \to T\ X$			$E \to T\ X$		
E'		$E' \to +\ T\ E'$			$E' \to \varepsilon$	$E' \to \varepsilon$
T	$T \to$ INT T'			$T \to (\ E\ )\ T'$		
T'		$T' \to \varepsilon$	$T' \to *\ T\ T'$, $T' \to \varepsilon$		$T' \to \varepsilon$	$T' \to \varepsilon$
X		$X \to E'$, $X \to +$ INT E'			$X \to E'$	$X \to E'$

3 The explanation for why *invalid-grammar.json* is invalid

The parsing table for *invalid-grammar.json* contains the following conflicts:

- The cell in row T' and column $*$ contains both $T' \to *\,T\,T'$ and $T' \to \varepsilon$. This is a FIRST/FOLLOW conflict: $*$ belongs to both FIRST$(*\,T\,T')$ and FOLLOW(T').
- The cell in row X and column + contains both $X \to E'$ and $X \to +\,\text{INT}\,E'$. This is a FIRST/FIRST conflict: + belongs to both FIRST(E') and FIRST$(+\,\text{INT}\,E')$.

These conflicts mean that an LL(1) parser cannot uniquely select a production using only the current nonterminal and one lookahead token. Therefore, the grammar is not LL(1) and is unsuitable for this LL(1) parser. Here, "invalid" refers to its suitability for LL(1) parsing, rather than its validity as a context-free grammar.

Although parsing table conflicts do not necessarily imply ambiguity, this grammar is also ambiguous: some strings have more than one distinct parse tree, as demonstrated below.

3.1 INT + INT

The string INT + INT can be derived using either production of X.

Using $X \to E'$:

$$\begin{aligned} E &\Rightarrow T\,X \\ &\Rightarrow \text{INT}\,T'\,X \\ &\Rightarrow \text{INT}\,X \\ &\Rightarrow \text{INT}\,E' \\ &\Rightarrow \text{INT} + T\,E' \\ &\Rightarrow \text{INT} + \text{INT}\,T'\,E' \\ &\Rightarrow \text{INT} + \text{INT}\,E' \\ &\Rightarrow \text{INT} + \text{INT}. \end{aligned}$$

Using $X \to + \text{INT}\,E'$:

$$\begin{aligned} E &\Rightarrow T\,X \\ &\Rightarrow \text{INT}\,T'\,X \\ &\Rightarrow \text{INT}\,X \\ &\Rightarrow \text{INT} + \text{INT}\,E' \\ &\Rightarrow \text{INT} + \text{INT}. \end{aligned}$$

These are two distinct leftmost derivations of the same string, so they correspond to different parse trees. Therefore, the grammar is ambiguous.

3.2 INT * INT * INT

The string $\text{INT} * \text{INT} * \text{INT}$ can also be derived in two different ways. In the production $T' \to * T\,T'$, the nested T can itself generate further multiplication through $T \to \text{INT}\,T'$. Consequently, the final multiplication can be generated either by the T' inside this nested T or by the T' following it.

Using the T' inside the nested T to generate the final multiplication:

$$\begin{aligned} E &\Rightarrow T\,X \\ &\Rightarrow \text{INT}\,T'\,X \\ &\Rightarrow \text{INT} * T\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT}\,T'\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT} * T\,T'\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}\,T'\,T'\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}\,T'\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}\,X \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}\,E' \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}. \end{aligned}$$

Using the T' following the nested T to generate the final multiplication:

$$\begin{aligned} E &\Rightarrow T\,X \\ &\Rightarrow \text{INT}\,T'\,X \\ &\Rightarrow \text{INT} * T\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT}\,T'\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT}\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT} * T\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}\,T'\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}\,T'\,X \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}\,X \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}\,E' \\ &\Rightarrow \text{INT} * \text{INT} * \text{INT}. \end{aligned}$$

These are also two distinct leftmost derivations of the same string. They place the final multiplication in different parts of the parse tree, providing another example of the grammar's ambiguity.